

Scoring, Term Weighting and the Vector Space Model

Academic Year 2025/2026

Luigi Santangelo
luigi.santangelo@unipv.it

University of Pavia
**Master Degree in
Computer Engineering**

Boolean vs Free Text Retrieval Model

All the methods described so far (Single Word Index, Bi- and Multi-Word Index, Positional Index, WildCard, Permuterm, K-Gram) are based on the **Boolean Retrieval Model**. Such model is able to retrieve all those documents containing all terms.

To help users finding all the information they need, Search Engines are also based on a different model, called **Free Text Retrieval Model**, which allows retrieving documents even though they do not contain all terms. Such models are based on **ranking**. The **Vector Space Model** supports ranking.

Boolean vs Free Text Retrieval Model

Feature	Boolean Retrieval	Free Text Retrieval
<i>Approach</i>	Logical Operators	Natural Language
<i>Precision</i>	High	Low
<i>Recall</i>	Lower	Higher
<i>Ranking</i>	Exact match	Relevance Based
<i>Easy to use</i>	Requires Learning	Simpler
<i>Result Scope</i>	Narrower	Broader
<i>Example</i>	Paris AND restaurant AND Michelin star	Best restaurants in Paris
<i>Suitable for</i>	Specific Information Retrieval	Information Exploration, Broader searches

The importance of using scores

Usually, the collection of documents retrieved after a search can be very large.

It is essential for a Search Engine to rank and order the documents matching a query.

To help user finding the right documents, Search Engines makes use of different ideas:

1. Parametric Index and Zone Index
2. Term Weighting
3. Vector Space Scoring

Parametric and Zone Index

Metadata

So far, we have considered a Document as a sequence of Terms. And for each Term in the dictionary we create a Posting List which is used during queries.

But a document is not just a sequence of terms, because sometimes it is **described by many other information**, such as the author, the date of creation, the title, the language

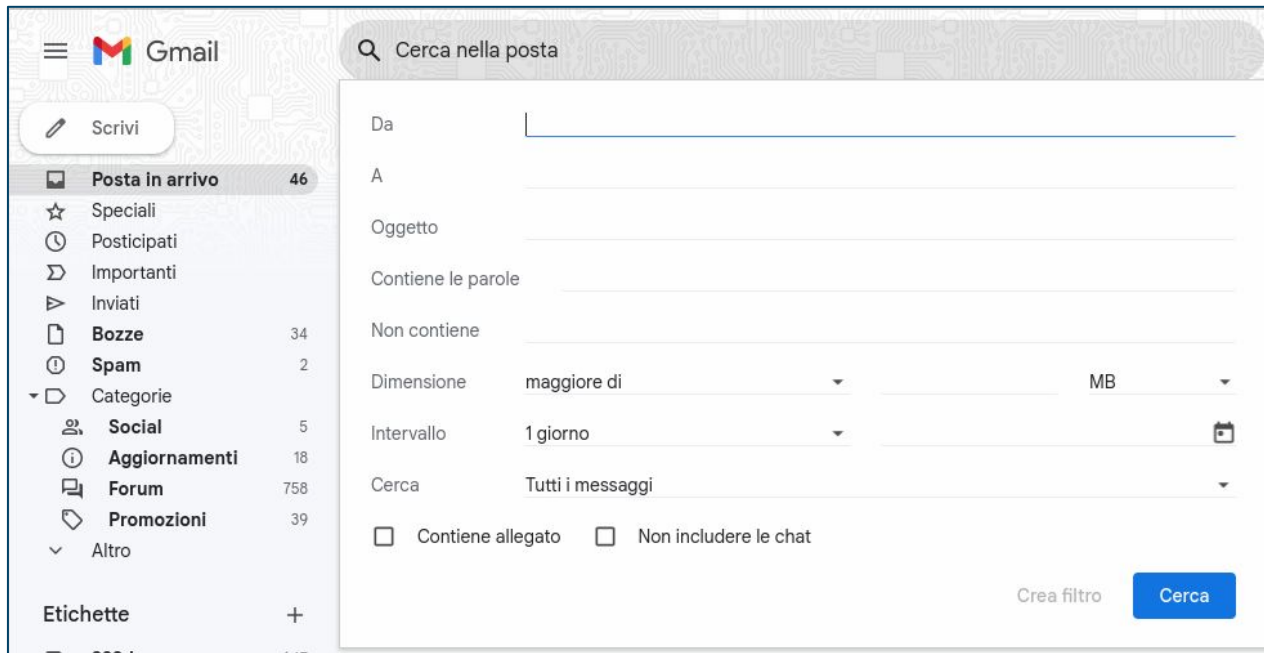
These data are known **Metadata**

Metadata

A metadata can be identified as:

- a **field**: if the value is limited within a set of possible alternatives (such as the author and the year of publishing)
- a **zone**: if the value is an arbitrary free text (such as abstract or title)

Using Metadata during Searches



The screenshot displays the Gmail search interface. On the left, the navigation sidebar includes the Gmail logo, a 'Scrivi' button, and a list of folders: 'Posta in arrivo' (46), 'Speciali', 'Posticipati', 'Importanti', 'Inviati', 'Bozze' (34), 'Spam' (2), 'Categorie' (expanded), 'Social' (5), 'Aggiornamenti' (18), 'Forum' (758), 'Promozioni' (39), and 'Altro'. Below these are 'Etichette' and a plus sign. The main search area has a header 'Cerca nella posta' with a magnifying glass icon. Below this, there are input fields for 'Da', 'A', and 'Oggetto'. There are also fields for 'Contiene le parole' and 'Non contiene'. The 'Dimensione' filter is set to 'maggiore di' and 'MB'. The 'Intervallo' filter is set to '1 giorno'. The 'Cerca' filter is set to 'Tutti i messaggi'. At the bottom, there are checkboxes for 'Contiene allegato' and 'Non includere le chat'. A 'Crea filtro' link and a blue 'Cerca' button are at the bottom right.

Gmail

Cerca nella posta

Scrivi

Posta in arrivo 46

- Speciali
- Posticipati
- Importanti
- Inviati
- Bozze** 34
- Spam** 2
- Categorie
 - Social** 5
 - Aggiornamenti** 18
 - Forum** 758
 - Promozioni** 39
- Altro

Etichette +

Da

A

Oggetto

Contiene le parole

Non contiene

Dimensione maggiore di MB

Intervallo 1 giorno

Cerca Tutti i messaggi

☐ Contiene allegato ☐ Non includere le chat

Crea filtro **Cerca**

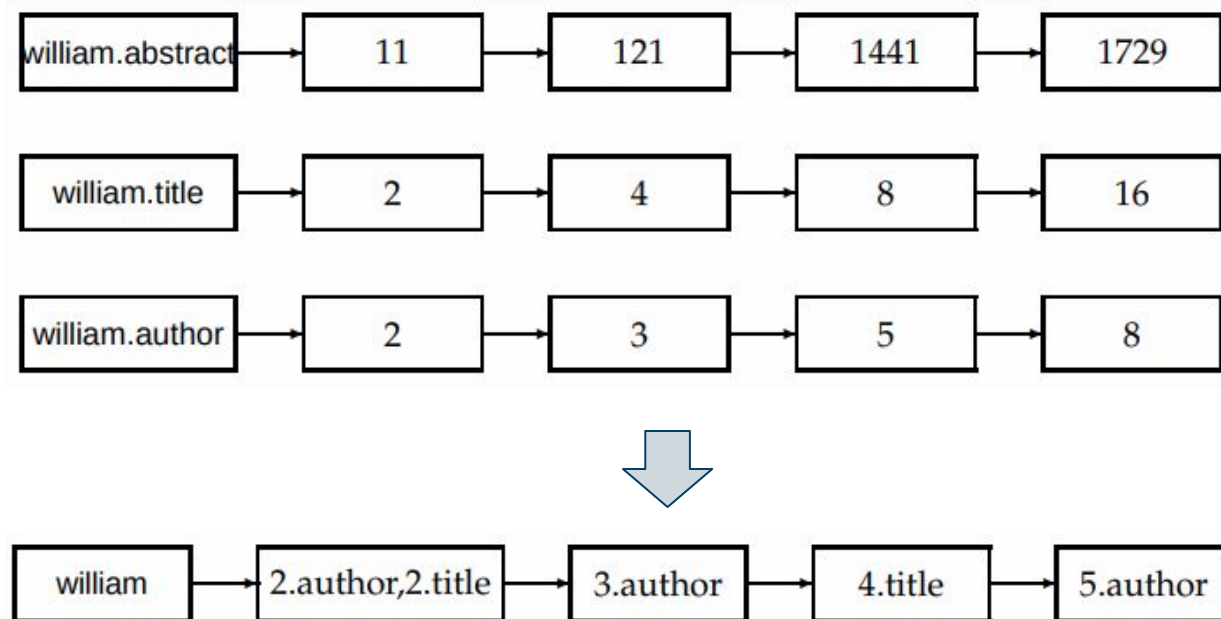
Parametric Index and Zone Index

A Parametric Index is a inverted index which is built using Field.

A Zone Index is a reverted index which is built using Zone.

During queries, Parametric Index, Zone Index and Inverted Index are merged together.

An example of Zone Index



Weighted Zone Scoring

Not all zones are equally important.

Search Engine should consider the weight of each zone.

Given a query term q and a document d , weighted zone scoring assigns to the pair (d, q) a score in the interval $[0, 1]$, by computing a **linear combination of zone scores**

Weighted Zone Scoring

Let's consider l zones. Each zone is assigned to a weight g_i where

$$g_i \in [0, 1] \text{ such that } \sum_{i=1}^l g_i = 1$$

Let's consider also $s_i(\mathbf{d}, \mathbf{q}) \in \{0, 1\}$ where 0 means that the term does not occur in the zone i of the document d , 1 if it does. Then the **weighted zone scoring** can be computed as

$$score(d, q) = \sum_{i=1}^l g_i * s_i(d, q)$$

Weighted Zone Scoring

For example: Let's suppose to have 3 zones: *author*, *title*, *body*, having weight respectively $g_1=0.2$, $g_2=0.3$ and $g_3=0.5$. We want to search for the term $q = \textit{shakespeare}$.

Now, consider a document d where the term appears only on the title and the body. So, the score of this document would be **$0.2*0 + 0.3*1 + 0.5*1 = 0.8$**

How to use the right score for the zones?

An trivial way to assign scores is by asking the user (or an expert)

But assigning the right scores can be viewed as a simple **optimization problem** and we can learn the weights.

Let's start simple. Suppose we have a collection of documents each of them described by two zones: *title* and *body*.

How to use the right score for the zones?

Given a query term q and a document d , we can compute $s_T(d, q)$ and $s_B(d, q)$

both functions will give 0 or 1 depending on the matching of the query to the title and to the body.

If we had the weights for both zones, we could easily compute the score, but we do not have such weights, we need to learn them:

$$score(d, q) = g * s_T(d, q) + (1 - g) * s_B(d, q)$$

How to use the right score for the zones?

Now, our aim is to determine the constant g from **a set of training examples** built by a human editor.

Such set is made by quintuples of the form $\Phi_j = (d, q, s_T(d, q), s_B(d, q), r(d, q))$ where:

- d is the document
- q is the query term
- $s_T(d, q) = 1$ if the title in d contains q , 0 otherwise
- $s_B(d, q) = 1$ if the body in d contains q , 0 otherwise
- $r(d, q)$ is the **relevance judgment** assessed by the human editor,
 - **1 for *relevant*** and **0 for *non relevant***.

How to use the right score for the zones?

Example	DocID	Query	s_T	s_B	Judgment
Φ_1	37	linux	1	1	Relevant
Φ_2	37	penguin	0	1	Non-relevant
Φ_3	238	system	0	1	Relevant
Φ_4	238	penguin	0	0	Non-relevant
Φ_5	1741	kernel	1	1	Relevant
Φ_6	2094	driver	0	1	Relevant
Φ_7	3191	driver	1	0	Non-relevant

How to use the right score for the zones?

Now we can compute the difference between the predicted score and the real score as

$$\epsilon(g, \Phi_j) = (r(d_j, q_j) - \textit{score}(d_j, q_j))^2$$

This is the error of the scoring function with weight g . We can compute the total error as

$$\sum_j \epsilon(g, \Phi_j)$$

We want to make the error as **the minimum**

How to use the right score for the zones?

Now, let's split Φ in 8 different parts:

 Φ_{00r} Φ_{00n}

$$\Phi = \Phi_{00r} \cup \Phi_{01r} \cup \Phi_{10r} \cup \Phi_{11r} \cup \Phi_{00n} \cup \Phi_{01n} \cup \Phi_{10n} \cup \Phi_{11n}$$

 Φ_{01r} Φ_{01n} Φ_{10r} Φ_{10n}

Now, let's compute the error for one element belonging each of these subsets

 Φ_{11r} Φ_{11n}

How to use the right score for the zones?

and then consider the cardinality of each subset

- n_{00r} and n_{00n} : number of training example Φ_j where $s_T(d_j, q_j) = 0$ and $s_B(d_j, q_j) = 0$ that are relevant and non relevant respectively
- n_{01n} and n_{01n} : number of training example Φ_j where $s_T(d_j, q_j) = 0$ and $s_B(d_j, q_j) = 1$ that are non relevant and non relevant respectively
- n_{10r} and n_{10n} : number of training example Φ_j where $s_T(d_j, q_j) = 1$ and $s_B(d_j, q_j) = 0$ that are non relevant and non relevant respectively
- n_{11r} and n_{11n} : number of training example Φ_j where $s_T(d_j, q_j) = 1$ and $s_B(d_j, q_j) = 1$ that are non relevant and non relevant respectively

How to use the right score for the zones?

Let's compute the error for one item belonging to Φ_{00r}

$$\epsilon(g, \Phi_{00r_j}) = [r(d_j, q_j) - \text{score}(d_j, q_j)]^2$$

$$\epsilon(g, \Phi_{00r_j}) = [r(d_j, q_j) - (gs_T(d_j, q_j) + (1 - g)s_B(d_j, q_j))]^2$$

$$\epsilon(g, \Phi_{00r_j}) = [1 - (g * 0 + (1 - g) * 0)]^2 = 1$$

How to use the right score for the zones?

Let's compute the error for one item belonging to Φ_{00n}

$$\epsilon(g, \Phi_{00n_j}) = [r(d_j, q_j) - \text{score}(d_j, q_j)]^2$$

$$\epsilon(g, \Phi_{00n_j}) = [r(d_j, q_j) - (gs_T(d_j, q_j) + (1 - g)s_B(d_j, q_j))]^2$$

$$\epsilon(g, \Phi_{00n_j}) = [0 - (g * 0 + (1 - g) * 0)]^2 = 0$$

How to use the right score for the zones?

Let's compute the error for one item belonging to Φ_{01r}

$$\epsilon(g, \Phi_{01r_j}) = [r(d_j, q_j) - \text{score}(d_j, q_j)]^2$$

$$\epsilon(g, \Phi_{01r_j}) = [r(d_j, q_j) - (gs_T(d_j, q_j) + (1 - g)s_B(d_j, q_j))]^2$$

$$\epsilon(g, \Phi_{01r_j}) = [1 - (g * 0 + (1 - g) * 1)]^2 = g^2$$

How to use the right score for the zones?

Let's compute the error for one item belonging to Φ_{01n}

$$\epsilon(g, \Phi_{01n_j}) = [r(d_j, q_j) - \text{score}(d_j, q_j)]^2$$

$$\epsilon(g, \Phi_{01n_j}) = [r(d_j, q_j) - (gs_T(d_j, q_j) + (1 - g)s_B(d_j, q_j))]^2$$

$$\epsilon(g, \Phi_{01n_j}) = [0 - (g * 0 + (1 - g) * 1)]^2 = (1 - g)^2$$

How to use the right score for the zones?

Let's compute the error for one item belonging to Φ_{10r}

$$\epsilon(g, \Phi_{10r_j}) = [r(d_j, q_j) - \text{score}(d_j, q_j)]^2$$

$$\epsilon(g, \Phi_{10r_j}) = [r(d_j, q_j) - (gs_T(d_j, q_j) + (1 - g)s_B(d_j, q_j))]^2$$

$$\epsilon(g, \Phi_{10r_j}) = [1 - (g * 1 + (1 - g) * 0)]^2 = (1 - g)^2$$

How to use the right score for the zones?

Let's compute the error for one item belonging to Φ_{10n}

$$\epsilon(g, \Phi_{10n_j}) = [r(d_j, q_j) - \text{score}(d_j, q_j)]^2$$

$$\epsilon(g, \Phi_{10n_j}) = [r(d_j, q_j) - (gs_T(d_j, q_j) + (1 - g)s_B(d_j, q_j))]^2$$

$$\epsilon(g, \Phi_{10n_j}) = [0 - (g * 1 + (1 - g) * 0)]^2 = g^2$$

How to use the right score for the zones?

Let's compute the error for one item belonging to Φ_{11r}

$$\epsilon(g, \Phi_{11r_j}) = [r(d_j, q_j) - \text{score}(d_j, q_j)]^2$$

$$\epsilon(g, \Phi_{11r_j}) = [r(d_j, q_j) - (gs_T(d_j, q_j) + (1 - g)s_B(d_j, q_j))]^2$$

$$\epsilon(g, \Phi_{11r_j}) = [1 - (g * 1 + (1 - g) * 1)]^2 = 0$$

How to use the right score for the zones?

Let's compute the error for one item belonging to Φ_{11n}

$$\epsilon(g, \Phi_{11n_j}) = [r(d_j, q_j) - \text{score}(d_j, q_j)]^2$$

$$\epsilon(g, \Phi_{11n_j}) = [r(d_j, q_j) - (gs_T(d_j, q_j) + (1 - g)s_B(d_j, q_j))]^2$$

$$\epsilon(g, \Phi_{11n_j}) = [0 - (g * 1 + (1 - g) * 1)]^2 = 1$$

How to use the right score for the zones?

Let's now compute the total error

$$\begin{aligned} \sum_{j \in \Phi} \epsilon(g, \Phi_j) = & \sum_{j \in \Phi_{00n}} \epsilon(g, \Phi_{00r_j}) + \sum_{j \in \Phi_{00n}} \epsilon(g, \Phi_{00n_j}) + \sum_{j \in \Phi_{01n}} \epsilon(g, \Phi_{01r_j}) + \sum_{j \in \Phi_{01n}} \epsilon(g, \Phi_{01n_j}) + \\ & \sum_{j \in \Phi_{10n}} \epsilon(g, \Phi_{10r_j}) + \sum_{j \in \Phi_{10n}} \epsilon(g, \Phi_{10n_j}) + \sum_{j \in \Phi_{11n}} \epsilon(g, \Phi_{11r_j}) + \sum_{j \in \Phi_{11n}} \epsilon(g, \Phi_{11n_j}) \end{aligned}$$

How to use the right score for the zones?

Let's now compute the total error

$$\sum_{j \in \Phi} \epsilon(g, \Phi_j) = n_{00r} * 1 + n_{00n} * 0 + n_{01r} * g^2 + n_{01n} * (1 - g)^2 + \\ n_{10r} * (1 - g)^2 + n_{10n} * g^2 + n_{11r} * 0 + n_{11n} * 1$$

$$\sum_{j \in \Phi} \epsilon(g, \Phi_j) = (1 - g)^2(n_{01n} + n_{10r}) + g^2(n_{01r} + n_{10n}) + n_{00r} + n_{11n}$$

How to use the right score for the zones?

As we want to make the error as minimum as possible, let's compute the derivative of the error

$$\begin{aligned} d[(1 - g)^2(n_{01n} + n_{10r}) + g^2(n_{01r} + n_{10n}) + n_{00r} + n_{11n}] = \\ - 2(1 - g)(n_{01n} + n_{10r}) + 2g(n_{01r} + n_{10n}) \end{aligned}$$

How to use the right score for the zones?

Setting the derivative to zero we can get

$$-2(1 - g)(n_{01n} + n_{10r}) + 2g(n_{01r} + n_{10n}) = 0$$

$$g = \frac{n_{01n} + n_{10r}}{n_{01n} + n_{10r} + n_{01r} + n_{10n}}$$

Term Frequency and Weighting

Weighted Zone Scoring

So far, we have just assigned a score depending on whether or not a term appears in a document.

$$score(d, q) = \sum_{i=1}^l g_i * s_i(d, q)$$

Let's consider also $s_i(d, q) \in \{0, 1\}$ where 0 means that the term does not occur in the zone i of the document d , 1 if it does.

Term Count, Term Frequency, Document Frequency

Sometimes the importance of a document might depend on the frequency a term appear in the collection:

Three scores are usually used:

- **Term Count**
- **Term Frequency**
- **Document Count**
- **Document Frequency**

Term Count, Term Frequency, Document Frequency

The **Term Count** $tc_{t,d}$ is the number of time a term t appears in a document d

The **Term Frequency** $tf_{t,d}$ is the number of time a term t appears in a document d over the number of words in the document d

$$tf_{t,d} = \frac{tc_{t,d}}{\sum_{\tau \in d} tc_{\tau,d}}$$

The **Document Count** d_t is is the number of documents a term appears in

The **Document Frequency** df_t is the number of documents a term appears in over the total number of documents in the collection

Using Term Frequency as Score

We can then use the Term Frequency for computing the score:

$$score(d, q) = tf_{t,d}$$

which can be easily extended if we have ***l* zones**:

$$score(d, q) = \sum_{i=1}^l g_i * tf_{t,d}^i$$

Term Frequency vs Inverse Document Frequency

Using a score based on the term frequency is not always a good idea, because documents containing several occurrences of that term **will have an higher score**.

The idea is to **normalize** the Term Frequency with the **Inverse Document Frequency**.

Inverse Document Frequency

$$idf_t = \log_{10} \frac{N}{d_t}$$

where

- N is the number of documents in the collection
- d_t is the number of documents containing the term t .

This means that the *idf* of a rare term is high, whereas the *idf* of a frequent term is likely to be low. The Inverse Document Frequency is **a measure of how much information the term provides**

Scaling the Term Frequency

Now we can scale the Term Frequency according to the Document Frequency as follows:

$$tf_idf_{t,d} = tf_{t,d} * idf_t$$

$tf_idf_{t,d}$ is

- highest when t occurs many time within a small number of documents
- lower when t occurs few time in a document
- 0 when t occur in all documents or the term does not appear at all in the document

An example

	Number of terms	Number of time term “dog” appears	tf	idf	tf_idf
<i>Document 01</i>	100	5	0,050	0,000	0,000
<i>Document 02</i>	100	5	0,050	0,000	0,000
<i>Document 03</i>	100	5	0,050	0,000	0,000
<i>Document 04</i>	100	5	0,050	0,000	0,000
<i>Document 05</i>	100	5	0,050	0,000	0,000
<i>Document 06</i>	100	5	0,050	0,000	0,000
<i>Document 07</i>	100	5	0,050	0,000	0,000
<i>Document 08</i>	100	5	0,050	0,000	0,000
<i>Document 09</i>	100	5	0,050	0,000	0,000
<i>Document 10</i>	100	5	0,050	0,000	0,000

An example

	Number of terms	Number of time term “dog” appears	tf	idf	tf_idf
<i>Document 01</i>	100	95	0,950	0,000	0,000
<i>Document 02</i>	100	95	0,950	0,000	0,000
<i>Document 03</i>	100	95	0,950	0,000	0,000
<i>Document 04</i>	100	95	0,950	0,000	0,000
<i>Document 05</i>	100	95	0,950	0,000	0,000
<i>Document 06</i>	100	95	0,950	0,000	0,000
<i>Document 07</i>	100	95	0,950	0,000	0,000
<i>Document 08</i>	100	95	0,950	0,000	0,000
<i>Document 09</i>	100	95	0,950	0,000	0,000
<i>Document 10</i>	100	95	0,950	0,000	0,000

An example

	Number of terms	Number of time term “dog” appears	tf	idf	tf_idf
<i>Document 01</i>	100	5	0,050	0,301	0,015
<i>Document 02</i>	100		0,000	0,301	0,000
<i>Document 03</i>	100	5	0,050	0,301	0,015
<i>Document 04</i>	100		0,000	0,301	0,000
<i>Document 05</i>	100	5	0,050	0,301	0,015
<i>Document 06</i>	100		0,000	0,301	0,000
<i>Document 07</i>	100	5	0,050	0,301	0,015
<i>Document 08</i>	100		0,000	0,301	0,000
<i>Document 09</i>	100	5	0,050	0,301	0,015
<i>Document 10</i>	100		0,000	0,301	0,000

An example

	Number of terms	Number of time term “dog” appears	tf	idf	tf_idf
<i>Document 01</i>	100	95	0,950	0,301	0,286
<i>Document 02</i>	100		0,000	0,301	0,000
<i>Document 03</i>	100	95	0,950	0,301	0,286
<i>Document 04</i>	100		0,000	0,301	0,000
<i>Document 05</i>	100	95	0,950	0,301	0,286
<i>Document 06</i>	100		0,000	0,301	0,000
<i>Document 07</i>	100	95	0,950	0,301	0,286
<i>Document 08</i>	100		0,000	0,301	0,000
<i>Document 09</i>	100	95	0,950	0,301	0,286
<i>Document 10</i>	100		0,000	0,301	0,000

An example

	Number of terms	Number of time term “dog” appears	tf	idf	tf_idf
<i>Document 01</i>	100	5	0,050	1,000	0,050
<i>Document 02</i>	100		0,000	1,000	0,000
<i>Document 03</i>	100		0,000	1,000	0,000
<i>Document 04</i>	100		0,000	1,000	0,000
<i>Document 05</i>	100		0,000	1,000	0,000
<i>Document 06</i>	100		0,000	1,000	0,000
<i>Document 07</i>	100		0,000	1,000	0,000
<i>Document 08</i>	100		0,000	1,000	0,000
<i>Document 09</i>	100		0,000	1,000	0,000
<i>Document 10</i>	100		0,000	1,000	0,000

An example

	Number of terms	Number of time term “dog” appears	tf	idf	tf_idf
<i>Document 01</i>	100	95	0,950	1,000	0,950
<i>Document 02</i>	100		0,000	1,000	0,000
<i>Document 03</i>	100		0,000	1,000	0,000
<i>Document 04</i>	100		0,000	1,000	0,000
<i>Document 05</i>	100		0,000	1,000	0,000
<i>Document 06</i>	100		0,000	1,000	0,000
<i>Document 07</i>	100		0,000	1,000	0,000
<i>Document 08</i>	100		0,000	1,000	0,000
<i>Document 09</i>	100		0,000	1,000	0,000
<i>Document 10</i>	100		0,000	1,000	0,000

An example: to recap

Conditions	tf_idf
Term appearing a few time in all documents	0.000
Term appearing many times in all documents	0.000
Term appearing a few time in few documents	0.015
Term appearing many times in few documents	0.286
Term appearing a few time in one document	0.050
Term appearing many times in one document	0.950
Term not appearing at all	0.000

The new score

So, we can change the way we score a document on a query term. That is, instead of summing up the number of occurrences of each term t in query q , we can use the tf_idf weight of each term in the document d :

$$score(d, q) = \sum_{t \in q} \sum_{i=1}^l g_i * tf_idf_{t,d}$$

The BM25 score function

BM25 (Best Match 25) is one ranking algorithm that has gained significant attention and success.

BM25 is a term-based ranking model that aims to provide accurate and relevant search results by scoring documents based on their term frequencies and document lengths.

It is currently used by many Search Engine, such as Solr and Elasticsearch

The BM25 score function

$$score(q, d) = \sum_{i=1}^{|q|} idf(q_i) \cdot \frac{tf(q_i, d) \cdot (k_1 + 1)}{tf(q_i, d) + k_1 \cdot (1 - b + b \cdot \frac{|d|}{avgdl})}$$

$$idf(q_i) = \log \frac{N - df(q_i) + 0.5}{df(q_i) + 0.5}$$

q_i = i^{th} term of the query

tf = term frequency (number of times a particular term appears in a document)

|d| = length of the document in words (terms)

avgdl = the average document length over all the documents of the collection

k_1 , **b** = free parameters, usually chosen as $k_1 = 2.0$ and $b = 0.75$

N = total number of documents in the collection

df(q_i) = the number of documents containing the query term q_i

Vector Space

Vector Space Model

The Vector Space Model supports **free text retrieval**, in which a query is specified as a set of words **without any query operators** connecting them.

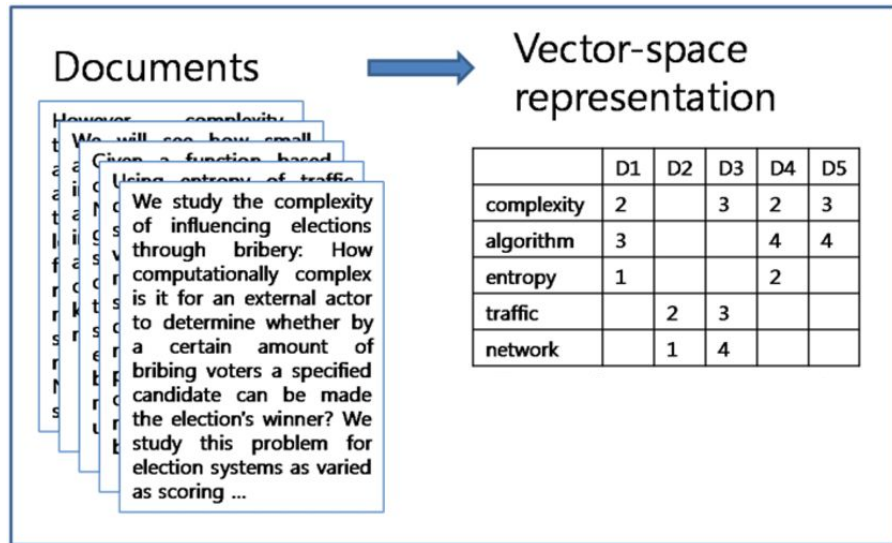
Documents matching the query are **scored** and **ranked**.

The interpretation of such free text queries is that **at least one of the query terms** be present in any retrieved document.

Vector Space Model

A document can be represented by a Vector $V(d)$ with one component for each dictionary term.

The component of the vector are computed using the **tc**, **tf** or **tf_idf** weighting schema



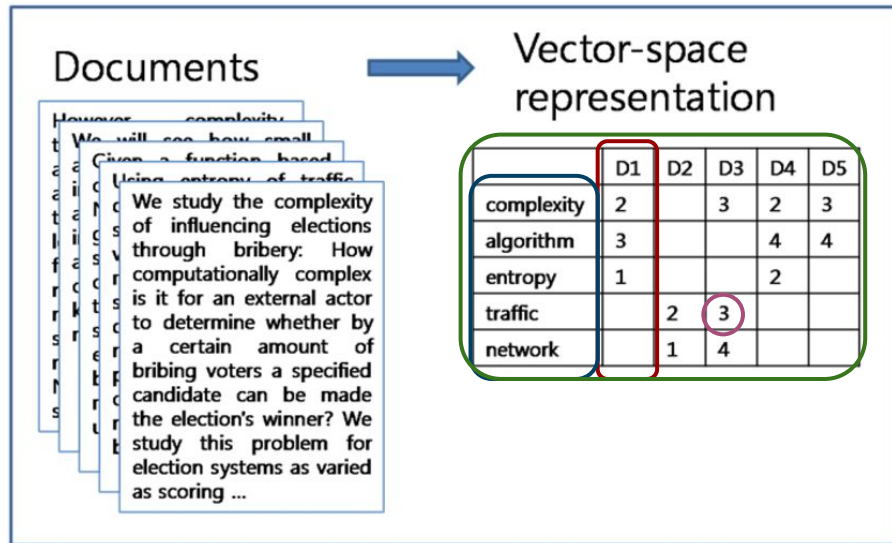
Vector Space Model

Term-Document Matrix

Vocabulary

tc, tf or tf_idf weight

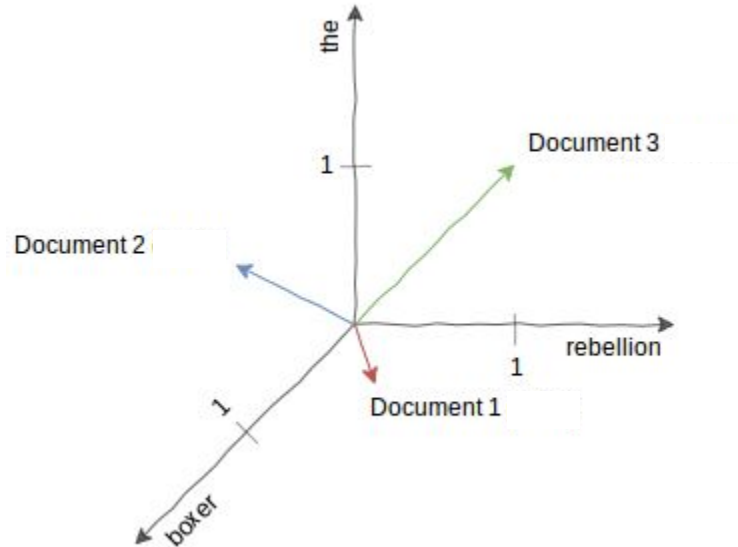
Vector representation for the document



Vector Space Model

We can represent graphically each document by using its own Vector.

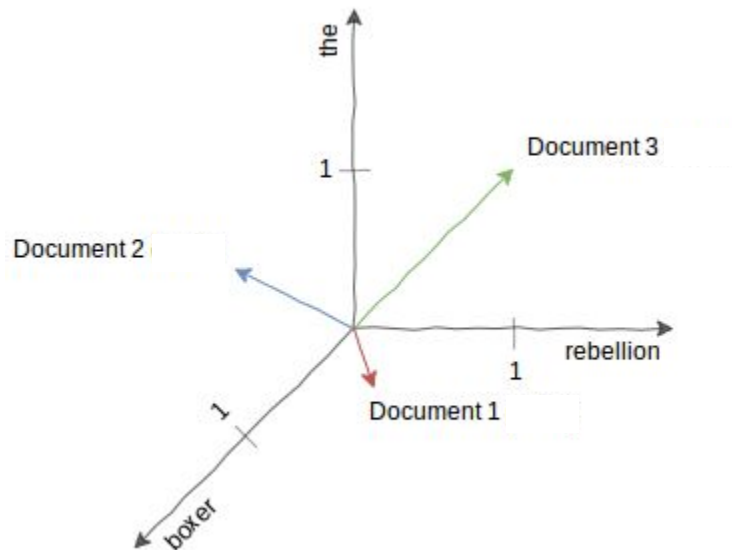
The axis of the space are the terms of the vocabulary. The arrows represent documents



Vector Space Model

Important: with the vector representation we lose the relative ordering of the terms in the documents. This means that the documents “*Mary is quicker than John*” and “*John is quicker than Mary*” are identical in this Vector Space Model.

For this reason this model is also well known in literature as **Bag of Words**



Weaknesses of the Bag of Words model

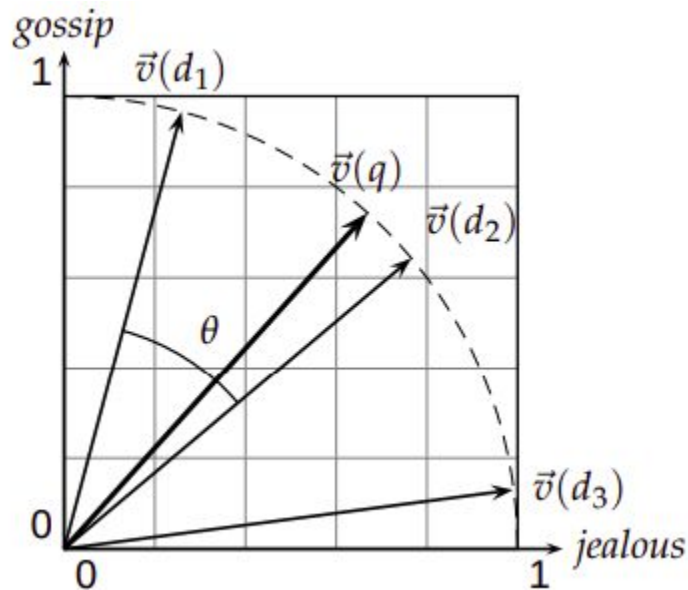
1. BoW treats all words the same, regardless of their order or meaning in a sentence ("time flies" and "flies time" would be represented identically, even though they have opposite meanings)
2. BoW does not take into account the context (the meaning) of each document
3. BoW creates very large vectors (having the same size of the vocabulary)

Solution: **Word Embedding** and **Document Embedding**

Cosine Similarities

We can measure the **similarity between two documents** just using their Vector Model.

Indeed, if the angle of two vectors, $\vec{v}(d_1)$ and $\vec{v}(d_2)$ is equal to 0, both vectors are identical and then both documents are identical (ignoring the order of the terms in the documents)



Cosine Similarities

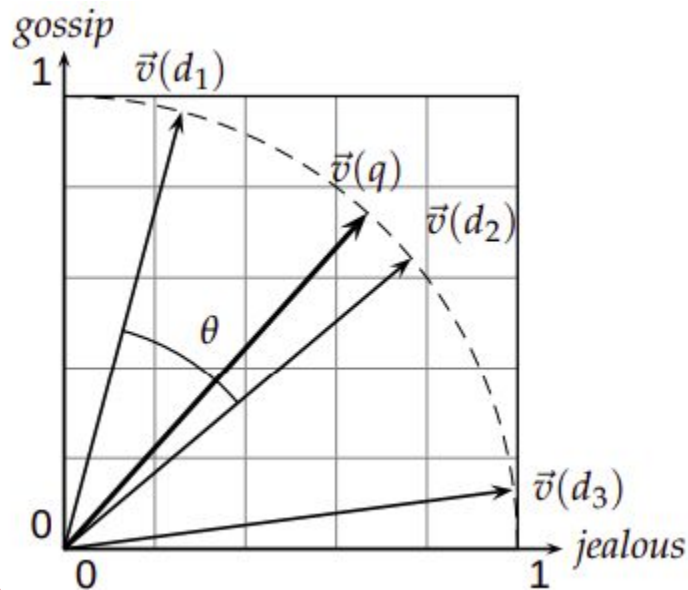
So, the smaller the angle, the more similar are the documents.

How can we compute similarity between two documents?

$$\text{sim}(d_1, d_2) = \cos\theta = \frac{\vec{V}(d_1) \cdot \vec{V}(d_2)}{|\vec{V}(d_1)| |\vec{V}(d_2)|}$$

norm 1

scalar product

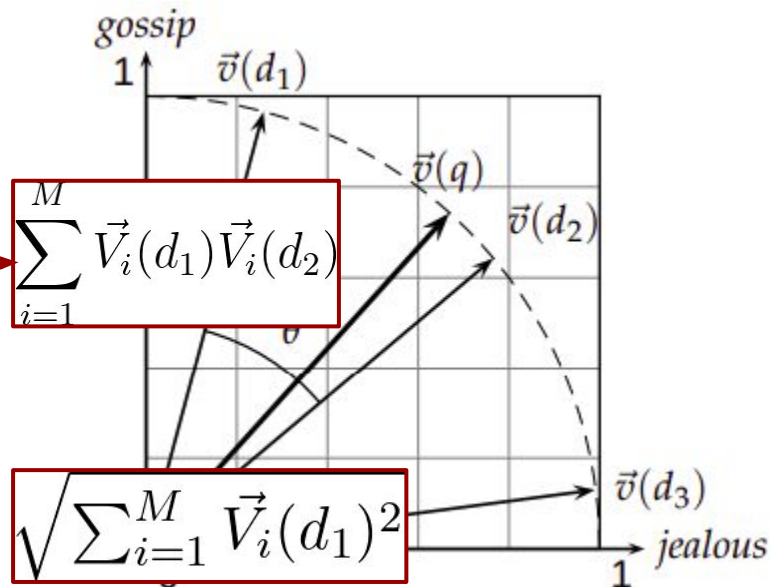


Cosine Similarities

So, the smaller the angle, the more similar are the documents.

How can we compute similarity between two documents?

$$\text{sim}(d_1, d_2) = \cos\theta = \frac{\vec{V}(d_1) \cdot \vec{V}(d_2)}{|\vec{V}(d_1)| |\vec{V}(d_2)|}$$



A real example

Let's suppose we have a collection of three different documents:

- d1: music is a universal language
- d2: music is a miracle
- d3: music is a universal feature of the human experience

Term Count Vector

Term	d1	d2	d3
<i>a</i>	1	1	1
<i>experience</i>	0	0	1
<i>feature</i>	0	0	1
<i>human</i>	0	0	1
<i>is</i>	1	1	1
<i>language</i>	1	0	0
<i>miracle</i>	0	1	0
<i>music</i>	1	1	1
<i>of</i>	0	0	1
<i>the</i>	0	0	1
<i>universal</i>	1	0	1
TOTAL	5	4	9

A real example

Term Count Vector

Term	d1	d2	d3
<i>a</i>	1	1	1
<i>experience</i>	0	0	1
<i>feature</i>	0	0	1
<i>human</i>	0	0	1
<i>is</i>	1	1	1
<i>language</i>	1	0	0
<i>miracle</i>	0	1	0
<i>music</i>	1	1	1
<i>of</i>	0	0	1
<i>the</i>	0	0	1
<i>universal</i>	1	0	1
TOTAL	5	4	9

$$tf_{t,d} = \frac{tc_{t,d}}{\sum_{\tau \in d} tc_{\tau,d}}$$



Term Frequency Vector

Term	d1	d2	d3
<i>a</i>	0,200	0,250	0,111
<i>experience</i>	0,000	0,000	0,111
<i>feature</i>	0,000	0,000	0,111
<i>human</i>	0,000	0,000	0,111
<i>is</i>	0,200	0,250	0,111
<i>language</i>	0,200	0,000	0,000
<i>miracle</i>	0,000	0,250	0,000
<i>music</i>	0,200	0,250	0,111
<i>of</i>	0,000	0,000	0,111
<i>the</i>	0,000	0,000	0,111
<i>universal</i>	0,200	0,000	0,111

A real example

Term Count Vector

Term	d1	d2	d3
<i>a</i>	1	1	1
<i>experience</i>	0	0	1
<i>feature</i>	0	0	1
<i>human</i>	0	0	1
<i>is</i>	1	1	1
<i>language</i>	1	0	0
<i>miracle</i>	0	1	0
<i>music</i>	1	1	1
<i>of</i>	0	0	1
<i>the</i>	0	0	1
<i>universal</i>	1	0	1
TOTAL	5	4	9

$$idf_t = \log_{10} \frac{N}{d_t}$$



IDF

Term	IDF
<i>a</i>	0,000
<i>experience</i>	0,477
<i>feature</i>	0,477
<i>human</i>	0,477
<i>is</i>	0,000
<i>language</i>	0,477
<i>miracle</i>	0,477
<i>music</i>	0,000
<i>of</i>	0,477
<i>the</i>	0,477
<i>universal</i>	0,176

A real example

Term Frequency Vector

Term	d1	d2	d3
a	0,200	0,250	0,111
experience	0,000	0,000	0,111
feature	0,000	0,000	0,111
human	0,000	0,000	0,111
is	0,200	0,250	0,111
language	0,200	0,000	0,000
miracle	0,000	0,250	0,000
music	0,200	0,250	0,111
of	0,000	0,000	0,111
the	0,000	0,000	0,111
universal	0,200	0,000	0,111

IDF

Term	IDF
a	0,000
experience	0,477
feature	0,477
human	0,477
is	0,000
language	0,477
miracle	0,477
music	0,000
of	0,477
the	0,477
universal	0,176

$$tf_idf_{t,d} = tf_{t,d} * idf_f$$



TF_IDF

Term	d1	d2	d3
a	0,000	0,000	0,000
experience	0,000	0,000	0,053
feature	0,000	0,000	0,053
human	0,000	0,000	0,053
is	0,000	0,000	0,000
language	0,095	0,000	0,000
miracle	0,000	0,119	0,000
music	0,000	0,000	0,000
of	0,000	0,000	0,053
the	0,000	0,000	0,053
universal	0,035	0,000	0,020

A real example

Term	d1	d2	d3
<i>a</i>	0,000	0,000	0,000
<i>experience</i>	0,000	0,000	0,053
<i>feature</i>	0,000	0,000	0,053
<i>human</i>	0,000	0,000	0,053
<i>is</i>	0,000	0,000	0,000
<i>language</i>	0,095	0,000	0,000
<i>miracle</i>	0,000	0,119	0,000
<i>music</i>	0,000	0,000	0,000
<i>of</i>	0,000	0,000	0,053
<i>the</i>	0,000	0,000	0,053
<i>universal</i>	0,035	0,000	0,020

Let's compute the similarity between d1 and d3:

$$\text{sim}(d_1, d_2) = \cos\theta = \frac{\vec{V}(d_1) \cdot \vec{V}(d_2)}{|\vec{V}(d_1)| |\vec{V}(d_2)|}$$

= 0.056

A real example

Term	d1	d2	d3
<i>a</i>	0,000	0,000	0,000
<i>experience</i>	0,000	0,000	0,053
<i>feature</i>	0,000	0,000	0,053
<i>human</i>	0,000	0,000	0,053
<i>is</i>	0,000	0,000	0,000
<i>language</i>	0,095	0,000	0,000
<i>miracle</i>	0,000	0,119	0,000
<i>music</i>	0,000	0,000	0,000
<i>of</i>	0,000	0,000	0,053
<i>the</i>	0,000	0,000	0,053
<i>universal</i>	0,035	0,000	0,020

Let's compute the similarity between d1 and d2:

$$\text{sim}(d_1, d_2) = \cos\theta = \frac{\vec{V}(d_1) \cdot \vec{V}(d_2)}{|\vec{V}(d_1)| |\vec{V}(d_2)|}$$

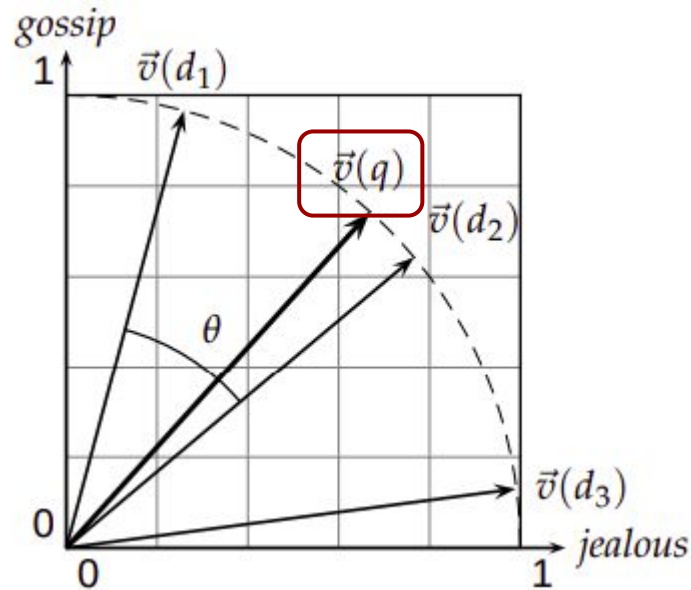
= 0.000

Queries as vectors

As the same as the documents, even **queries** can be considered like a document and then represented by a vector.

Then we can use the cosine similarity between the query vector and a document vector as a measure of the score of the document for that query:

$$score(d, q) = \frac{\vec{V}(q) \cdot \vec{V}(d)}{|\vec{V}(q)| |\vec{V}(d)|}$$



Queries as vectors

A document may have a high cosine score for a query even if it does not contain all query terms.

Queries as vectors

Once the score has been computed between the query and all documents belonging to the collection, **all documents are sorted depending on the score** and then the first K are retrieved.

For many search engines, usually $K = 10$

Computing the cosine similarities between the query vector and each document vector in the collection, sorting the resulting scores and selecting the top K documents **can be expensive**. Will see how to deal with this problem.

Variant tf_idf functions: $\text{wf_idf}_{t,d}$

Many other ways of defining weight have been defined.

Instead of using $\text{tf_idf}_{t,d}$ as weight we can use the **$\text{wf_idf}_{t,d}$**

$$\text{wf-idf}_{t,d} = \text{wf}_{t,d} \times \text{idf}_t$$
$$\text{wf}_{t,d} = \begin{cases} 1 + \log \text{tf}_{t,d} & \text{if } \text{tf}_{t,d} > 0 \\ 0 & \text{otherwise} \end{cases}$$

the logarithm of the term frequency

Variant tf_idf functions: $ntf_idf_{t,d}$

Another variant:

Instead of using $tf_idf_{t,d}$ as weight we can use the **$ntf_idf_{t,d}$**

$$ntf_idf_{t,d} = ntf_{t,d} \times idf_t$$

The diagram illustrates the calculation of $ntf_{t,d}$ from the equation $ntf_{t,d} = a + (1 - a) \frac{tf_{t,d}}{tf_{max}(d)}$. A blue arrow points from the $ntf_{t,d}$ term in the first equation to the $ntf_{t,d}$ term in the second equation. The variable a in the second equation is enclosed in a pink box, with a pink arrow pointing to it from the text "smoothing (0.4 or 0.5)". The fraction $\frac{tf_{t,d}}{tf_{max}(d)}$ is enclosed in a green box, with a green arrow pointing to it from the text "normalization". The denominator $tf_{max}(d)$ is enclosed in a red box, with a red arrow pointing to it from the text "maximum term frequency in the document".

$$ntf_{t,d} = a + (1 - a) \frac{tf_{t,d}}{tf_{max}(d)}$$

smoothing (0.4 or 0.5)

normalization

maximum term frequency in the document

Measuring Distances

Measuring distances

Jaccard Similarity Coefficient measures how much two different sets are similar to each other. Sometimes, instead, we are more interested to study how much two sets (or vectors) differ from each other.

Of course Jaccard Similarity can be used to get the **Jaccard Distance**, but it is important to notice that there are many distances measures, but all of them must have some properties:

Measuring distances

Definition: suppose we have a set of point, called space. Let x and y be two points. A distance measure on this between two points is a function $d(x, y)$ which gives a real number representing the distance between both points. Such a function must satisfy these properties:

1. $d(x, y) \geq 0$ [no negative distances]
2. $d(x, y) = 0$ iif $x = y$
3. $d(x, y) = d(y, x)$ [symmetric]
4. $d(x, y) \leq d(x, z) + d(z, y)$ [triangle inequality]

Measuring distances

- Jaccard Distance
- Manhattan and Euclidean Distance
- Cosine Distance
- Edit Distance
- Hamming Distance

Jaccard Distance

Let x and y be two n -dimensional vectors

$$d(x, y) = 1 - J(x, y)$$

Manhattan and Euclidean Distance

Let x and y two n -dimensional vectors

$$d([x_1, x_2, \dots, x_n], [y_1, y_2, \dots, y_n]) = \sqrt[r]{\sum_{i=1}^n |x_i - y_i|^r}$$

if $r = 1$ (L_1 norm): Manhattan Distance

if $r = 2$ (L_2 norm): Euclidean Distance

if $r = \inf$ ($L_{\inf}$ norm): defined as the maximum of $|x_i - y_i|$ over all dimensions i .

Cosine Similarity and Cosine Distance

cosine similarity = $S_C(A, B) := \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}},$

norm 1 points to $\|\mathbf{A}\|$

scalar product points to $\mathbf{A} \cdot \mathbf{B}$

cosine distance = $D_C(A, B) := 1 - S_C(A, B).$

Cosine Similarity and Cosine Distance

An example: Let $x = [1, 2, 1]$ and $y = [2, 1, 1]$ two vectors.

Let's compute the cosine similarity:

$$\frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}, \quad [1*2 + 2*1 + 1*1] / \{ \sqrt{1^2 + 2^2 + 1^2} * \sqrt{2^2 + 1^2 + 1^2} \} =$$
$$3 / \{ \sqrt{6} * \sqrt{6} \} = 3/6 = \frac{1}{2}$$

Then the cosine distance is: $1 - \frac{1}{2} = \frac{1}{2}$

Edit Distance

This distance makes sense when points are strings. The Edit Distance between two strings $x = x_1x_2\dots x_n$ and $y = y_1y_2\dots y_m$ is the smallest number of insertions and deletions of single characters that will convert x to y .

Ad example: $x = abcde$, $y = acfdeg$. To convert x to y we need:

- delete b
- insert f after c
- insert g after e

The Edit distance between x and y is 3

Edit Distance

There is another way for computing Edit Distance. It is based on the **Longest Common Subsequence (LCS)**. An LCS of x and y is a string that is constructed by deleting positions from x and y.

The edit distance can be calculated as the length of x plus the length of y minus twice the length of their LCS.

An example: x = abcde, y = acfdeg. The LCS = acde

Edit Distance = $\text{lenght}(x) + \text{lenght}(y) - 2 * \text{lenght}(\text{LCS}(x, y)) = 5 + 6 - 2 * 4 = 3$

Hamming Distance

The Hamming Distance between two vectors is the number of component they differ to each other. This kind of distance is particularly useful when data are **represented by binary values**

An example: $x = [1, 0, 1, 0, 1]$ and $y = [1, 1, 1, 1, 0]$

The hamming distance is 3